

Encrypted Payload Integration Guide (iOS / Android)

Audience: mobile developers implementing request/response body encryption against the existing ASP.NET Core API. **Scope:** byte-level spec, worked examples with known-good output, copy-pasteable Kotlin and Swift, error handling, checklist.

1. Overview

The API and the Angular web frontend already exchange bodies in an encrypted envelope. Your app has to speak the same scheme or every request will be rejected.

Why it exists: payload confidentiality at the application layer. TLS terminates at load balancers, reverse proxies, WAFs, and corporate MITM appliances; anything past that point sees plaintext bodies. Encrypting the body means the JSON stays opaque in proxy logs, APM traces, crash dumps, and on-disk request captures.

Caveat — read this before you build anything on top of it:

- This is **defense in depth, not a replacement for HTTPS**. Every call must still go over TLS. Do not disable certificate validation, do not allow cleartext HTTP, do not treat "the body is encrypted anyway" as a reason to relax transport security.
- Headers, URLs, query strings, status codes, and body *length* are **not** protected by this scheme. Don't put secrets in a query string and assume this covers you.
- CBC provides confidentiality only — **no authentication or integrity**. A tampered ciphertext usually fails padding, but that is not a MAC and must not be treated as one. Integrity comes from TLS.
- The 32-byte key ships inside your app binary. Anyone who can pull your APK/IPA can extract it. Treat this scheme as raising the cost of casual interception, not as protection against a determined attacker who controls the device. Store the key as described in §8, and raise it with the backend team if your threat model needs more than that.

2. The scheme in one screen

Parameter	Value
Algorithm	AES-256-CBC
Padding	PKCS7 (identical to PKCS5 for 16-byte blocks — JCE calls it <code>PKCS5Padding</code>)

Parameter	Value
Key	32-byte UTF-8 string, used directly as raw key bytes . No KDF, no hashing, no derivation.
IV	16 random bytes, freshly generated for every encryption operation
Plaintext	UTF-8 bytes of the JSON body
Output encoding	Base64 the IV and the ciphertext separately , join with a single ASCII colon: <code><ivBase64>:<cipherTextBase64></code>
Envelope	<code>{ "payload": "<ivBase64>:<cipherTextBase64>" }</code> — for both request and response
Content-Type	<code>application/json</code>

The key is **provided by the backend team via a secure channel**. It is not in this document, not in the repo, and not in your CI logs.

What is NOT encrypted

Pass these through untouched — do not wrap them in an envelope:

- Requests with no body (`GET`, `HEAD`, `DELETE` without a body).
- `multipart/form-data` requests (file uploads).
- Any response that is not `application/json` (binary downloads, redirects, empty `204`s).

Error responses are normally encrypted — a `400` still arrives as `{ "payload": "..."`. Handle that in your interceptor, or you'll surface "unknown error" for every validation failure. Infrastructure-level errors (502 from the gateway, HTML error pages) are not, so branch on content type and shape rather than status code alone.

3. How to encrypt a request

Given the plaintext JSON string and the shared key:

1. Serialize your request object to a JSON string.
2. `plaintextBytes = UTF8(jsonString)`
3. `keyBytes = UTF8(sharedKeyString)` → assert `keyBytes.length == 32`. Fail loudly if not; a 31- or 33-byte key silently produces garbage on some platforms and throws on others.

4. Generate `iv = 16` bytes from a **cryptographically secure** RNG (`SecureRandom`/`SecRandomCopyBytes`). Never a counter, never a constant, never `Random()`.
5. `cipherBytes = AES-256-CBC-Encrypt(key = keyBytes, iv = iv, data = PKCS7Pad(plaintextBytes))` — the platform Cipher applies the padding for you when you request `PKCS7`/`PKCS5Padding`.
6. `ivB64 = Base64Encode(iv)` (standard alphabet, with `=` padding, **no line breaks**)
7. `cipherB64 = Base64Encode(cipherBytes)`
8. `payload = ivB64 + ":" + cipherB64`
9. Send body: `{"payload": "<payload>"}` with `Content-Type: application/json`.

Base64 must be the **standard** alphabet (`(+)`, `(/)`), not URL-safe (`(-)`, `(_)`), and must not wrap lines. On Android this means `Base64.NO_WRAP`; the default `Base64.DEFAULT` inserts newlines and will break the server-side parse.

Worked example — request

Use these exact values to verify your implementation. Same key for both examples.

Key (UTF-8, 32 chars): 0123456789abcdef0123456789abcdef

IV (UTF-8, 16 chars): AbCdEfGhIjKlMnOp

IV (hex): 4162436445664768496a4b6c4d6e4f70

IV (base64): QWJDZEVmR2hJaktsTW5PcA==

Plaintext JSON (35 bytes):

```
{"userId":42,"action":"getProfile"}
```

After PKCS7 padding (48 bytes, hex):

```
7b22757365724964223a34322c226163744696f6e223a22676574450726f66696c65227d0d0d0d0d
```

Ciphertext (48 bytes, hex):

b0602c2940684ca7214aff8996875dfd328e4f55b8e59c81d49fd72aa4aec1c7d40178f1e8b6b45

Ciphertext (base64):

sGAsKUBoTKchSv+Jlodd/TKOT1W45ZyB1J/XKgSuwcfUAXjx6La0WNpmQXH1ISzi

Expected `(payload)` string:

QWJDZEVmR2hJaktSTW5PcA==:sGAsKUBoTKchSv+Jlodd/TKOT1W45ZyB1J/XKqSuwcfUAXjx6La0WN

Expected wire body:

```
json
```

```
{"payload": "QWJDZEVmR2hJaktsTW5PcA==:sGAsKUBoTKchSv+Jlodd/TKOT1W45ZyB1J/XKqSuwc
```

Note the plaintext is 35 bytes → padded to 48 with 13 bytes of `0x0d`. If your padded length is 35 or 40, your Cipher isn't applying PKCS7. If your ciphertext is 32 bytes, you're probably encrypting a truncated string.

The IV here is a readable ASCII string **only so you can reproduce it deterministically**. In production the IV is always random bytes. Hardcoding this IV is an immediate finding in review.

4. How to decrypt a response

1. Parse the response body as JSON, read the `payload` string field. If the field is absent, treat the body as already-plaintext or as an infrastructure error — do not attempt decryption.
2. Split on the **first** `:` into exactly two parts. Reject if there aren't two non-empty parts.
3. `iv = Base64Decode(part0)` → `assert(iv.length == 16)`.
4. `cipherBytes = Base64Decode(part1)` → `assert(length > 0)` and `length % 16 == 0`.
5. `plaintextBytes = AES-256-CBC-Decrypt(key = keyBytes, iv = iv, data = cipherBytes)` (Cipher strips PKCS7 padding).
6. `jsonString = UTF8Decode(plaintextBytes)`
7. Parse `jsonString` into your response model.

Steps 3 and 4 are cheap sanity checks that convert a class of malformed-input crashes into clean errors. Do them.

Worked example — response

```
Key (UTF-8):    0123456789abcdef0123456789abcdef
IV  (UTF-8):    ZyXwVuTsRqPoNmLk
IV  (base64):    Wn1Yd1Z1VHNScVBvTm1Maw==
```

Incoming body:

```
json
```

```
{"payload": "Wn1Yd1Z1VHNScVBvTm1Maw==:R9ZV0rsXY1ZVNnZ0DSqK5Qr4668yBQTb1x1RSEHk6q
```

Decrypts to (37 bytes):

json

```
{"status":"ok","data":{"name":"Ada"}}
```

Round-trip check: encrypting that plaintext with that key and IV must reproduce the ciphertext above, byte for byte.

5. Android / Kotlin

`javax.crypto`, no third-party dependency.

kotlin

```

import android.util.Base64
import java.nio.charset.StandardCharsets
import java.security.SecureRandom
import javax.crypto.Cipher
import javax.crypto.spec.IvParameterSpec
import javax.crypto.spec.SecretKeySpec

class PayloadCryptoException(message: String, cause: Throwable? = null) :
    Exception(message, cause)

object PayloadCrypto {

    private const val TRANSFORMATION = "AES/CBC/PKCS5Padding" // == PKCS7 for A
    private const val IV_LENGTH = 16
    private const val KEY_LENGTH = 32
    private const val B64 = Base64.NO_WRAP // never Base64.DEFAULT - it inserts

    private val secureRandom = SecureRandom()

    private fun secretKey(key: String): SecretKeySpec {
        val keyBytes = key.toByteArray(StandardCharsets.UTF_8)
        require(keyBytes.size == KEY_LENGTH) {
            "AES key must be exactly $KEY_LENGTH UTF-8 bytes, got ${keyBytes.si
        }
        return SecretKeySpec(keyBytes, "AES")
    }

    /** Returns "<ivBase64>:<cipherTextBase64>". */
    fun encrypt(plainJson: String, key: String): String {
        try {
            val iv = ByteArray(IV_LENGTH).also { secureRandom.nextBytes(it) }

            val cipher = Cipher.getInstance(TRANSFORMATION)
            cipher.init(Cipher.ENCRYPT_MODE, secretKey(key), IvParameterSpec(iv

            val cipherBytes = cipher.doFinal(plainJson.toByteArray(StandardChar

            return Base64.encodeToString(iv, B64) + ":" +
                Base64.encodeToString(cipherBytes, B64)
        } catch (e: Exception) {
            throw PayloadCryptoException("Failed to encrypt request payload", e
        }
    }
}

```

```

/** Accepts "<ivBase64>:<cipherTextBase64>", returns the plaintext JSON str
fun decrypt(payload: String, key: String): String {
    val parts = payload.split(":", limit = 2)
    if (parts.size != 2 || parts[0].isEmpty() || parts[1].isEmpty()) {
        throw PayloadCryptoException("Malformed payload: expected '<iv>:<ci
    }

    val iv: ByteArray
    val cipherBytes: ByteArray
    try {
        iv = Base64.decode(parts[0], B64)
        cipherBytes = Base64.decode(parts[1], B64)
    } catch (e: IllegalArgumentException) {
        throw PayloadCryptoException("Malformed payload: invalid base64", e
    }

    if (iv.size != IV_LENGTH) {
        throw PayloadCryptoException("Malformed payload: IV is ${iv.size} b
    }
    if (cipherBytes.isEmpty() || cipherBytes.size % 16 != 0) {
        throw PayloadCryptoException("Malformed payload: ciphertext length
    }

    try {
        val cipher = Cipher.getInstance(TRANSFORMATION)
        cipher.init(Cipher.DECRYPT_MODE, secretKey(key), IvParameterSpec(iv
        return String(cipher.doFinal(cipherBytes), StandardCharsets.UTF_8)
    } catch (e: Exception) {
        // BadPaddingException / IllegalBlockSizeException land here – almo
        throw PayloadCryptoException("Failed to decrypt response payload",
    }
}
}

```

OkHttp interceptor

kotlin

```

import okhttp3.*
import okhttp3.MediaType.Companion.toMediaType
import okhttp3.RequestBody.Companion.toRequestBody
import okio.Buffer
import org.json.JSONObject

class EncryptedPayloadInterceptor(private val keyProvider: () -> String) : Interceptor {

    private val jsonMediaType = "application/json; charset=utf-8".toMediaType()

    override fun intercept(chain: Interceptor.Chain): Response {
        val key = keyProvider()
        val original = chain.request()
        val body = original.body

        val request = when {
            body == null -> original // GET
            body.contentType()?.type == "multipart" -> original // multipart
            else -> {
                val plain = Buffer().also { body.writeTo(it) }.readUtf8()
                val envelope = JSONObject()
                    .put("payload", PayloadCrypto.encrypt(plain, key))
                    .toString()
                original.newBuilder()
                    .method(original.method, envelope.toRequestBody(jsonMediaType))
                    .build()
            }
        }

        val response = chain.proceed(request)
        val responseBody = response.body ?: return response
        if (responseBody.contentType()?.subtype != "json") return response

        val raw = responseBody.string()
        if (raw.isEmpty()) return response

        val payload = runCatching { JSONObject(raw).optString("payload", "") }.getOrNull()
        if (payload?.isEmpty() == true) {
            // Not an envelope (gateway error page, plain error, 204) – hand it as is
            return response.newBuilder()
                .body(ResponseBody.create(responseBody.contentType(), raw))
                .build()
        }
    }
}

```



```

        val plain = PayloadCrypto.decrypt(payload, key)
        return response.newBuilder()
            .body(ResponseBody.create(responseBody.contentType(), plain))
            .build()
    }
}

```

`body.writeTo(Buffer())` is safe for one-shot buffering here; if you also use a logging interceptor, place it **below** this one in the chain so it only ever sees ciphertext.

6. iOS / Swift

CryptoKit will not work for this. CryptoKit exposes `AES.GCM` and `ChaChaPoly` (both AEAD) and, since iOS 17, `AES.KeyWrap` — it has no public raw CBC API with a caller-supplied IV. GCM ciphertext is **not** interoperable with CBC. Use **CommonCrypto** (system framework, zero dependencies) or **CryptoSwift**. CommonCrypto is shown below and is the recommended option.

`import CommonCrypto` works directly from Swift 5 / iOS 13+; no bridging header is needed for a normal app target. In a Swift Package you'll need a small system-library target or an `@_implementationOnly` shim.

```

swift

```

```

import Foundation
import CommonCrypto

enum PayloadCryptoError: Error {
    case invalidKeyLength(Int)
    case malformedPayload(String)
    case decryptionFailed(status: Int32)
    case encryptionFailed(status: Int32)
    case invalidUTF8
}

enum PayloadCrypto {

    private static let ivLength = kCCBlockSizeAES128    // 16
    private static let keyLength = kCCKeySizeAES256      // 32

    // MARK: Encrypt

    /// Returns "<ivBase64>:<cipherTextBase64>".
    static func encrypt(_ plainJson: String, key: String) throws -> String {
        let keyData = Data(key.utf8)
        guard keyData.count == keyLength else {
            throw PayloadCryptoError.invalidKeyLength(keyData.count)
        }

        var iv = Data(count: ivLength)
        let rngStatus = iv.withUnsafeMutableBytes { ptr in
            SecRandomCopyBytes(kSecRandomDefault, ivLength, ptr.baseAddress!)
        }
        guard rngStatus == errSecSuccess else {
            throw PayloadCryptoError.encryptionFailed(status: rngStatus)
        }

        let cipherData = try crypt(operation: CCOperation(kCCEncrypt),
                                   input: Data(plainJson.utf8),
                                   key: keyData,
                                   iv: iv)

        return iv.base64EncodedString() + ":" + cipherData.base64EncodedString()
    }

    // MARK: Decrypt

```

```

/// Accepts "<ivBase64>:<cipherTextBase64>", returns the plaintext JSON str
static func decrypt(_ payload: String, key: String) throws -> String {
    let keyData = Data(key.utf8)
    guard keyData.count == keyLength else {
        throw PayloadCryptoError.invalidKeyLength(keyData.count)
    }

    // split on the FIRST colon only
    guard let sep = payload.firstIndex(of: ":") else {
        throw PayloadCryptoError.malformedPayload("missing ':' separator")
    }
    let ivPart = String(payload[payload.startIndex..

```

```

let status: CCCryptorStatus = output.withUnsafeMutableBytes { outPtr in
    input.withUnsafeBytes { inPtr in
        key.withUnsafeBytes { keyPtr in
            iv.withUnsafeBytes { ivPtr in
                CCCrypt(operation,
                        CCAAlgorithm(kCCAlgorithmAES),
                        CCOptions(kCCOptionPKCS7Padding), // PKCS7; d
                        keyPtr.baseAddress, key.count,
                        ivPtr.baseAddress,
                        inPtr.baseAddress, input.count,
                        outPtr.baseAddress, outputCapacity,
                        &outMoved)

            }
        }
    }
}

guard status == kCCSuccess else {
    throw operation == CCOperation(kCCEncrypt)
        ? PayloadCryptoError.encryptedFailed(status: status)
        : PayloadCryptoError.decryptedFailed(status: status)
}

output.removeSubrange(outMoved..

```

Passing `kCCOptionPKCS7Padding` **without** `kCCOptionECBMode` gives you CBC with the IV you supply — that's the combination you need. A bad-padding decrypt returns `kCCDecodeError` (`-4304`) rather than trapping.

URLSession usage

```
swift
```

```

struct Envelope: Codable { let payload: String }

func encryptedBody<T: Encodable>(_ value: T, key: String) throws -> Data {
    let json = String(data: try JSONEncoder().encode(value), encoding: .utf8)!
    let envelope = Envelope(payload: try PayloadCrypto.encrypt(json, key: key))
    return try JSONEncoder().encode(envelope)
}

func decodeEncrypted<T: Decodable>(_ type: T.Type, from data: Data, key: String
// Not an envelope? Fall back to plain decoding (204s, gateway errors, mult
guard let envelope = try? JSONDecoder().decode(Envelope.self, from: data) e
    return try JSONDecoder().decode(type, from: data)
}
let json = try PayloadCrypto.decrypt(envelope.payload, key: key)
return try JSONDecoder().decode(type, from: Data(json.utf8))
}

```

CryptoSwift alternative

If you'd rather not touch CommonCrypto:

```

swift

import CryptoSwift

let aes = try AES(key: Array(key.utf8), blockMode: CBC(iv: ivBytes), padding: .
let cipherBytes = try aes.encrypt(Array(plainJson.utf8))

```

Same wire format. Slower than CommonCrypto and adds a dependency; fine for low request volumes.

7. Error handling

Decryption failures are a normal runtime condition — key rotation, a partial response, a captive-portal login page returned as JSON. **Never let one crash the app** and never `try!` / `!!` a crypto call.

Condition	Likely cause	Client behaviour
<code>payload</code> field missing	Not an envelope: gateway error, <code>204</code> , HTML error page	Don't decrypt. Handle as a normal HTTP error using the status code.
No <code>:</code> separator, empty half	Truncated or corrupted body	Generic "Something went wrong, please try again." Retry once at most.
Base64 decode failure	Corrupted transport, wrong Base64 alphabet on send	Same as above. Check you're using <code>NO_WRAP</code> / standard alphabet.
IV $\neq$ 16 bytes, ciphertext not a multiple of 16	Malformed payload	Same as above.
<code>BadPaddingException</code> / <code>kCCDecodeError (-4304)</code>	Almost always a key mismatch — wrong environment key, key rotated, or a whitespace/newline in your stored key	Non-retryable. Surface "Unable to secure connection to the server" and log the <i>error type only</i> .
Plaintext isn't valid UTF-8 or valid JSON	Decrypted with the right key but the server sent something unexpected	Treat as a server error; report to your crash reporter without the body.
Key not 32 bytes at startup	Config/build error	Fail fast at app start with a clear developer-facing message. This should never reach production.

Practical rules:

- Wrap every crypto call in a single typed error (`PayloadCryptoException` / `PayloadCryptoError`) and map it to **one** user-facing message. Users can't act on "bad padding".
- Distinguish retryable (transport corruption) from non-retryable (key mismatch). Retrying a key mismatch just burns battery.
- Send the **error type and endpoint** to your crash reporter. Never the payload, never the ciphertext, never the key.
- Add a startup self-test in debug builds: encrypt/decrypt round-trip a fixed string and assert it matches. Catches a bad key config before your first real request.

8. Non-goals / do not do this

- **Don't reuse an IV.** One fresh random IV per encryption call. Reusing an IV across requests leaks whether two plaintexts share a prefix and is the single most common way this scheme gets broken in practice. Generate inside the encrypt function so it's impossible to pass a stale one in.
- **Don't use a non-CSPRNG for the IV.** `java.util.Random`, `arc4random` seeded by hand, timestamps, or a counter are all wrong. `SecureRandom` and `SecRandomCopyBytes` only.
- **Don't derive the key from a password, device ID, or anything else client-side.** No PBKDF2, no SHA-256 of a passphrase, no salting. The key is used verbatim as 32 raw UTF-8 bytes. A derivation step here produces a different key than the server has and every request fails.
- **Don't hash, trim, pad, or re-encode the key.** Watch for a trailing newline if it's read from a file or an env var — that's a 33-byte key and it will fail with a padding error you'll spend an afternoon on.
- **Don't log the decrypted payload, the plaintext JSON, or the key.** Not in `Log.d`, not in `print`, not in an `OkHttp` `HttpLoggingInterceptor` at `BODY` level, not in Charles/Proxyman captures shared in tickets. Put logging interceptors *below* the crypto interceptor so they only see ciphertext, and strip verbose logging from release builds.
- **Don't hardcode the key in source.** Keep it out of git. Inject at build time (Gradle property → `BuildConfig`, `.xcconfig` → `Info.plist`) from CI secrets, and consider storing it in Keystore/Keychain after first launch. This is obfuscation, not real protection — see §1.
- **Don't switch to GCM, ECB, or a different padding "because it's better."** Both ends must match. GCM is genuinely better; changing it is a backend conversation, not a client-side decision.
- **Don't encrypt multipart bodies or empty-body requests.** The server won't expect an envelope and will reject them.
- **Don't disable TLS validation or allow cleartext traffic.** Keep `usesCleartextTraffic=false` and App Transport Security on.
- **Don't treat successful decryption as authentication.** It proves the sender had the key; it is not a signature and not a session.

9. Pre-ship checklist

Correctness

- ☐ Key loads as exactly **32 UTF-8 bytes**; asserted at startup, no trailing whitespace/newline
- ☐ Encrypt produces `<ivBase64>:<cipherTextBase64>`, standard Base64 alphabet, **no line wrapping** (`NO_WRAP` on Android)
- ☐ Request bodies sent as `{"payload": "..."}` with `Content-Type: application/json`
- ☐ Response `payload` split on the **first** colon only
- ☐ **Verified against the §3 worked example** — same key + same IV produces byte-identical ciphertext
- ☐ **Verified against the §4 worked example** — the given payload decrypts to the expected JSON
- ☐ Local round-trip test passes: encrypt → decrypt → original string, including non-ASCII and emoji
- ☐ End-to-end against a real endpoint: request accepted, response decoded into a model

IV handling

- ☐ IV is 16 bytes from `SecureRandom` / `SecRandomCopyBytes`
- ☐ IV generated **inside** the encrypt call, never passed in, never cached
- ☐ Two identical requests produce two different `payload` strings (easy way to prove no IV reuse)

Pass-through

- ☐ `GET` / bodyless requests not wrapped
- ☐ `multipart/form-data` uploads not wrapped
- ☐ Non-JSON and non-envelope responses handled without attempting decryption
- ☐ Encrypted **error** responses (`4xx` / `5xx` with an envelope) decrypt and surface the real server message

Error handling

- ☐ No `!!`, `try!`, or force-unwrap on any crypto or Base64 path
- ☐ Malformed payload → clean typed error, no crash
- ☐ Bad padding / key mismatch → clean typed error, non-retryable, clear user message
- ☐ Crash reports contain error type + endpoint only — **never** payload, ciphertext, or key

Security hygiene

- ☐ Key not in source control; injected from CI secrets at build time
- ☐ Body-level HTTP logging disabled in release, and any logging interceptor sits below the crypto interceptor
- ☐ No `Log` / `print` of plaintext JSON anywhere, including debug helpers

- ☐ TLS validation intact; cleartext traffic disabled; ATS enabled
- ☐ Tested against staging and production keys separately (a key mismatch is the #1 integration failure)